

项目创新说明

作品名称：基于深度强化学习的云一边一端异构计算资源管理调度方法。
参赛队伍：操作系统创新小分队。

本项目针对已知任务依赖和代价的静态、非抢占云边端 DAG 调度，提出以下三项项目层面的技术贡献。贡献集中在公开算法的场景适配、约束搜索和安全工程组合，不将 HEFT、PPO、非法动作屏蔽或 LNS 本身宣称为原创。初赛已包含 Residual 排序与 LNS 精修；决赛在此基础上增加结构感知候选、HEFT 锚点选优、容量安全、软预算及可核验 CLI 交付。

创新一：共享 EFT 物理规则的 HEFT 残差任务排序

最接近的已有工作与能力：HEFT 依据 upward rank 排序并以插入式最早完成时间 EFT 分配资源，提供快速且可解释的启发式方案（Topcuoglu 等，2002）；PPO 提供策略优化框架（Schulman 等，2017），MaskablePPO 提供合法动作屏蔽基础设施（Huang 与 Ontañón，2020）。这些基础方法分别具备构造调度和更新策略的能力，但需要具体 DAG 环境、观测、奖励和策略适配代码才能组合使用。

本项目新增贡献：将学习动作限制为当前就绪任务的排序，资源放置交由统一 EFT 重放器；在标准化 HEFT rank 上叠加 tanh 限幅的网络残差，并对残差输出层零初始化。策略的采样、动作评估和推理共用残差公式及 mask；环境、HEFT 和搜索共用插入函数，避免不同物理规则造成虚假性能差异。零初始化只保证初始确定性动作锚定 HEFT 的就绪优先顺序，不表示初始随机轨迹都等价于 HEFT。

对应代码：src/policies/residual_ranking_policy.py 中的 ResidualRankingPolicy 与 _residual_action_logits；
src/env/scheduling_env_ranked.py、src/env/scheduling_env_residual.py；
src/env/scheduling_utils.py 中的 find_earliest_slot；
src/baselines/heft_scheduler.py。训练配置为
training/configs/ppo_mlp_residual.yaml，输入容量 30，训练分布 8~25 任务，
checkpoint 为 training/checkpoints/ppo_mlp_residual.zip。

验证依据：tests/test_residual_scheduling.py、
tests/test_scheduling_consistency.py 分别检查残差先验与时间线一致性；
evaluation/results/final_pipeline_lns_summary.json 保存同一 checkpoint 的五轮配对初解。Residual Best-of-64 均值为 0.950814 ± 0.003358 ，说明学习与多次采样组合可构造优于 HEFT 的候选；该数值包含采样收益，不当作单次确定性网络独立收益，也不代表五次独立训练。历史奖励、归一化和动作空间失败记录保留在 evaluation/results/，用于说明设计取舍。

创新二：可重放验证的 DAG 合法重定位与破坏—修复精修

最接近的已有工作与能力：LNS 及 ALNS 利用破坏—修复扩大组合搜索范围，Ropke 与 Pisinger（2006）研究了车辆路径问题中的自适应大邻域搜索；本项目初赛 Residual Best-of-N 已可生成多个合法初解。通用 LNS 思想与增加采样均不能直接替代 DAG 前驱约束检查、资源放置重算和候选性能验收。

本项目新增贡献：为 DAG 实现单任务重定位、位置锚点释放与带优先队列的 Kahn 拓扑修复。重定位检查全部依赖边；LNS 随机释放 2~4 个任务的原位置优先级，修复后执行一轮重定位。每个候选重新按共享 EFT 规则生成完整时间线，仅接受 makespan 严格改善超过 10^{-12} 的方案，并保留当前最优解。搜索改变任务优先序，资源映射和起止时间重新计算，避免沿用失效时间线。该实现未采用按算子历史收益更新权重的经典 ALNS 机制。

对应代码：src/policies/residual_local_search_scheduler.py 中的 schedule_task_order、improve_task_order、destroy_and_repair_order、large_neighborhood_search；tests/test_residual_local_search_scheduler.py 检查拓合法性与 best-only 接受。搜索记录候选评估数、接受移动数、完成迭代数及超时状态，便于将质量与计算投入对应。

验证依据：
evaluation/results/autonomous_exploration/compute_matched_sampling/paired_comparison_summary.json 记录锁定工作量后的历史五轮配对对照：纯 Residual Best-of-128 为 0.942048 ± 0.002063 ，20 场景平均 56.802 秒；初赛 Residual 加重定位及 LNS 为 0.920890 ± 0.001729 ，平均 54.402 秒，配对差值 -0.021158 ± 0.001148 ，双侧 $p=2.072081 \times 10^{-6}$ 。这支持该固定协议下合法精修具有超出单纯增加采样的收益。此对照针对初赛 LNS，不能写成决赛 Adaptive 已在等耗时条件下击败 Best-of-128。

创新三：结构感知候选、安全容量降级与可核验调度交付

最接近的已有工作与能力：标准 HEFT 可在不同规模 DAG 上快速输出启发式调度；本项目初赛固定容量 Residual/LNS 系统具备学习初解和合法精修能力，但历史宽并行组 mean_ratio=1.011915、仅 2/5 优于 HEFT，说明单一训练分布存在结构短板，容量 30 的模型也不能直接接收任意规模输入。

本项目新增贡献：Adaptive 始终保留精确 HEFT 锚点，并按统一 makespan 从 Residual/LNS、HEFT/LNS 及结构感知候选中选优；层宽比例不小于 0.30 时，在预算允许下增加宽度感知和成本优先拓扑序。超过 30 任务时明确跳过模型、记录原因并继续启发式搜索。CLI 提供 fast、balanced、quality 工作量与软墙钟预算，分别记录模型载入、求解耗时、内存、候选来源和完整调度计划，最终再校验任务完整、资源互斥及通信就绪约束。同一模拟器下 HEFT 锚点提供候选选优下限，软预算允许原子计算完成，不构成硬实时期限保证。

对应代码：src/policies/residual_local_search_scheduler.py 中的 describe_dag_shape、width_aware_topological_order、ResidualLargeNeighborhoodScheduler.schedule；scripts/schedule.py、src/evaluation/schedule_validation.py；tests/test_portfolio_robustness.py、tests/test_schedule_cli.py。结果 JSON 的 selected_source、diagnostics、validation 与 schedule 用于检查实际选优路径和交付合法性。

验证依据：evaluation/results/final_pipeline_lns_summary.json 中最终五轮 mean_ratio=0.916441±0.000984，每轮 20/20 严格优于 HEFT；相对配对 Residual 初解差值-0.034373±0.003685，双侧 $p=3.121263\times 10^{-5}$ 。evaluation/results/wide_parallel_adaptive.json 中新版宽并行专项为 0.929848，4/5 严格改善、5/5 不劣于 HEFT；历史与专项输入不同，不以 1.011915 到 0.929848 作为同实例配对效应。evaluation/results/final_adaptive_large_scale.json 中 30~60 任务 20 场景在 10 秒软预算下为 0.941627±0.026569，20/20 严格改善、峰值 RSS358.1MiB；40~60 任务 15 场景明确跳过模型，验证完整系统安全扩展，不宣称神经网络完成规模外泛化。

适用边界：结论限于静态 DAG、固定资源代价和已有测试协议；多次采样与搜索以额外秒级计算换取 makespan 质量。五轮主统计使用同一 checkpoint 而非多训练种子；Docker 矩阵验证国产发行版用户态兼容性，共享 WSL2 内核，不等同于国产原生内核、多机或端设备部署。动态到达、故障恢复、公平性、能耗及真实集群执行仍属后续工作。

方法来源

Topcuoglu 等（2002），Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing, DOI:10.1109/71.993206；Schulman 等（2017），Proximal Policy Optimization Algorithms, <https://arxiv.org/abs/1707.06347>；Huang 与 Ontañón（2020），A Closer Look at Invalid Action Masking in Policy Gradient Algorithms, <https://arxiv.org/abs/2006.14171>；Ropke 与 Pisinger（2006），An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows, DOI:10.1287/trsc.1050.0135。